Práctica 3: Funciones, condicionales y bucles en R

Laboratorio de Matemáticas

Grado en Ingeniería Matemática - Doble Grado en Física e Ingeniería Matemática

Miguel Beltra Vidal

Curso 2026/27

1. Objetivos

El objetivo de esta práctica es introducir algunas de las estructuras fundamentales de programación en R. Al finalizarla, el estudiante será capaz de definir y utilizar funciones propias, utilizar estructuras condicionales mediante `if`, `else if` y `else`, utilizar bucles `for` y `while`, y utilizar `break` y `next` para controlar la ejecución de un bucle.

2. Mostrando texto por pantalla

En R, los **strings** o cadenas de texto se representan escribiendo el texto entre comillas simples (`'...'`) o dobles (`"..."`). Se pueden almacenar en variables y combinar con otros textos. Para mostrar información por consola se utiliza principalmente la función `print()`, que muestra el contenido de un objeto, o `cat()`, que permite construir mensajes combinando diferentes elementos de forma más flexible. Por ejemplo:

```
nombre <- "Ana"
edad <- 20

print(nombre)
print(edad)

cat("Hola", nombre, "tienes", edad, "años.\n")
```

En este caso, `print()` muestra cada objeto por separado, mientras que `cat()` permite crear un mensaje completo. El carácter `\n` representa un **salto de línea**.

3. Funciones

Una función es un bloque de código que realiza una determinada tarea. Las funciones permiten organizar el código y reutilizar instrucciones sin tener que escribirlas varias veces. En R, una función se define mediante la palabra reservada `function`. La estructura básica es:

```
nombre <- function(argumento1, argumento2) {
  instrucciones
}
```

Por ejemplo, podemos definir una función que calcule el cuadrado de un número:

```
cuadrado <- function(x) {
  x^2
}
```

Una vez definida, podemos utilizarla tantas veces como queramos:

```
> cuadrado(3)
```

La salida será:

```
[1] 9
```

También podemos almacenar el resultado en una variable:

```
> resultado <- cuadrado(7)
> resultado
```

Ejercicio 1. Define las siguientes funciones:

a) Una función `doble(x)` que devuelva el doble de un número.

b) Una función `mitad(x)` que devuelva la mitad de un número.

c) Una función `area_circulo(r)` que calcule el área de un círculo de radio `r`.

d) Una función `media(a,b,c)` que calcule la media aritmética de tres números.

e) Una función `hipotenusa(b,c)` que calcule la hipotenusa de un triángulo rectángulo cuyos catetos son `b` y `c`.

Comprueba cada una de las funciones con varios valores diferentes.

3.1. El valor que devuelve una función

En R, una función devuelve normalmente el resultado de la última expresión evaluada. Por ejemplo:

```
area_rectangulo <- function(base, altura) {
  area <- base * altura
  area
}
```

Podemos utilizarla de la siguiente manera:

```
> area_rectangulo(4, 7)
```

obteniendo:

```
[1] 28
```

También podemos utilizar explícitamente la instrucción `return()`:

```
area_rectangulo <- function(base, altura) {
  area <- base * altura
  return(area)
}
```

Las dos formas son válidas.

Ejercicio 2. Define una función `distancia(x1,y1,x2,y2)` que calcule la distancia euclídea entre dos puntos $P = (x_1, y_1)$ y $Q = (x_2, y_2)$. Recuerda que la distancia euclídea se define como:

$$d(P, Q) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}.$$

Calcula la distancia entre los puntos `(1,2)` y `(4,6)`.

3.2. Argumentos por defecto

Una función puede tener argumentos con valores por defecto. Por ejemplo:

```
potencia <- function(x, n = 2) {
  x^n
}
```

En este caso, si no especificamos `n`, se utiliza el valor 2:

```
> potencia(5)
```

produce:

```
[1] 25
```

Pero también podemos indicar otro exponente:

```
> potencia(5, 3)
```

que produce:

```
[1] 125
```

Ejercicio 3. Define una función `saludo(nombre = "estudiante")` que muestre un mensaje de bienvenida. Comprueba el funcionamiento de:

```
> saludo()
> saludo("Ana")
```

Debería salir:

```
[1] Hola estudiante, te damos la bienvenida
[1] Hola Ana, te damos la bienvenida
```

4. Estructuras condicionales

En muchas situaciones necesitamos que un programa realice una acción u otra dependiendo de que se cumpla una determinada condición. En  podemos utilizar la estructura:

```
if (condicion) {
  instrucciones
}
```

Por ejemplo:

```
x <- 7

if (x > 0) {
  print("El número es positivo")
}
```

Como $7 > 0$, el programa mostrará:

```
[1] "El número es positivo"
```

Si la condición no se cumple, no se ejecuta el código contenido dentro del `if`.

4.1. La estructura `if ... else`

Cuando queremos elegir entre dos posibilidades podemos utilizar `else`:

```
x <- -3

if (x >= 0) {
  print("El número es no negativo")
} else {
  print("El número es negativo")
}
```

Ejercicio 4. Escribe una función `signo(x)` que:

- devuelva `"positivo"` si $x > 0$;
- devuelva `"negativo"` si $x < 0$;
- devuelva `"cero"` si $x = 0$.

Comprueba el funcionamiento de la función para los valores

$$8,\ -3,\ 0,\ 15,\ -12.$$

4.2. Varias condiciones

Cuando tenemos más de dos posibilidades podemos utilizar `else if`. Por ejemplo:

```
nota <- 7.5

if (nota < 5) {
    print("Suspenso")
} else if (nota < 7) {
    print("Aprobado")
} else if (nota < 9) {
    print("Notable")
} else {
    print("Sobresaliente")
}
```

Observa que, aunque `nota` cumple varias condiciones, únicamente se ejecuta la primera el primer bloque para el que la condición es cierta.

Ejercicio 5. Define una función `calificacion(nota)` que reciba una nota numérica y muestre:

- `"Suspenso"` si la nota es menor que 5;
- `"Aprobado"` si está entre 5 y 7;
- `"Notable"` si está entre 7 y 9;
- `"Sobresaliente"` si es mayor o igual que 9.

Comprueba qué ocurre con los valores

$$3{,}5,\ 5,\ 6{,}9,\ 7,\ 8{,}5,\ 9,\ 10.$$

¿Qué ocurre si introducimos una nota negativa o una nota mayor que 10? Modifica la función para detectar esos casos.

4.3. Condiciones múltiples

Las condiciones pueden combinarse mediante operadores lógicos. Por ejemplo:

```
x <- 8
if (x >= 5 && x <= 10) {
  print("El número está entre 5 y 10")
}
```

También podemos utilizar el operador `||`:

```
x <- -2
if (x < 0 || x > 100) {
  print("El número está fuera del intervalo [0,100]")
}
```

Ejercicio 6. Escribe una función `en_intervalo(x,a,b)` que indique si `x` pertenece al intervalo cerrado $[a, b]$. Por defecto, la función deberá comprobar que `x` pertenece a $[0, 1]$. Por ejemplo:

```
> en_intervalo(5, 1, 10)
```

```
[1] TRUE
```

mientras que

```
> en_intervalo(15, 1, 10)
```

```
[1] FALSE
```

5. El bucle `for`

Los bucles permiten repetir un conjunto de instrucciones varias veces. La estructura básica de un bucle `for` es:

```
for (variable in secuencia) {
  instrucciones
}
```

Por ejemplo:

```
for (i in 1:5) {
  print(i)
}
```

produce:

```
[1] 1
[1] 2
[1] 3
[1] 4
[1] 5
```

También podemos utilizar un bucle para realizar cálculos:

```
for (i in 1:5) {
  print(i^2)
}
```

Ejercicio 7. Escribe funciones que utilicen un bucle `for` para:

a) Mostrar los números enteros del 1 al 20.

b) Mostrar los números pares entre 1 y 50.

c) Mostrar los cuadrados de los números del 1 al 10.

d) Mostrar las potencias de 2 desde 2^0 hasta 2^{10}.

e) Mostrar la tabla de multiplicar del 7.

5.1. Acumuladores

Los bucles son especialmente útiles para calcular sumas y productos. Por ejemplo, podemos calcular la suma de los primeros 10 números naturales mediante:

```
suma <- 0

for (i in 1:10) {
  suma <- suma + i
}

suma
```

El resultado será:

```
[1] 55
```

Observa que la variable `suma` va acumulando los resultados parciales.

Ejercicio 8. Utiliza bucles `for` para calcular:

a) La suma de los números del 1 al 100.

b) La suma de los números pares del 1 al 100.

c) El producto de los números del 1 al 10.

d) La suma $1^2 + 2^2 + 3^2 + \cdots + 100^2$.

e) La suma $\frac{1}{1} + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{100}$.

6. El bucle `while`

Otra estructura repetitiva disponible en R es `while`. Su estructura es:

```
while (condicion) {
  instrucciones
}
```

Las instrucciones se ejecutan mientras la condición sea verdadera. Por ejemplo:

```
i <- 1

while (i <= 5) {
  print(i)
  i <- i + 1
}
```

produce los números del 1 al 5.

Es importante modificar alguna variable relacionada con la condición dentro del bucle. De lo contrario, podemos crear un **bucle infinito**, esto es, un bucle que no termine.

Ejercicio 9. Utiliza un bucle `while` para:

a) Mostrar los números del 1 al 20.

b) Mostrar los números impares menores que 50.

c) Calcular la suma de los números del 1 al 100.

d) Calcular cuántas veces podemos dividir 1000 entre 2 antes de obtener un número menor que 1.

7. `break` y `next`

La instrucción `break` permite terminar inmediatamente un bucle. Por ejemplo:

```
for (i in 1:100) {
  if (i > 10) {
    break
  }
  print(i)
}
```

Aunque el bucle está definido hasta 100, termina cuando `i` supera 10.

La instrucción `next` permite saltar a la siguiente iteración:

```
for (i in 1:10) {
  if (i %% 2 == 0) {
    next
  }
  print(i)
}
```

Esta función muestra solamente los números impares.

Ejercicio 10.

a) Utiliza `break` para recorrer los números del 1 al 100 y detener el bucle cuando aparezca el primer múltiplo de 17 mayor que 50.

b) Utiliza `next` para mostrar los números del 1 al 30 excepto aquellos que sean múltiplos de 3.

c) Modifica el ejercicio anterior para mostrar únicamente los números que sean múltiplos de 3 pero no de 5.

8. Entrega y consejos

Sube tus ficheros a GitHub antes de la fecha acordada con el profesor. Revisa el flujo habitual de trabajo de Git que se encuentra al final del enunciado de la práctica 2. Acuérdate de seguir una estructura ordenada de directorios y ficheros. Comenta tus scripts para que en un futuro puedas acordarte de qué hace cada uno.

Idea clave: las funciones permiten reutilizar código, las estructuras condicionales permiten tomar decisiones y los bucles permiten repetir instrucciones.